

Experiment – 1.1.2

Student Name: Damandeep Singh

UID: 24BDA7O313

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 10-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit
- Browser: Google Chrome (for testing and rendering)

3. Objective:

Users should be able to save an in-progress post as a draft, retrieve any saved draft back into the composer for further editing, and delete drafts they no longer need — all persisted in frontend state (Redux) rather than lost on navigation. The system should optionally simulate the latency of a real backend save/fetch operation, so the interaction pattern matches what a networked API would look like.

4. Theory:

A draft management system needs three operations — create, update, and delete — each represented as a Redux action so the UI stays a pure function of state. Storing drafts in normalized form (`{ byId, allIds }`) means loading a draft back into the editor, or deleting one, is an $O(1)$ lookup rather than a linear search. To retrieve a draft, its full record (text, selected platforms, media, timestamps) is read back from the store and used to reinitialize the composer's local state. "Simulating backend interaction" refers to wrapping a save/delete action in an async thunk with an artificial delay (e.g. via `setTimeout` inside `createAsyncThunk`),

so the UI can show a pending state exactly as it would when waiting on a real network request — useful for demonstrating the pattern without needing an actual server for this experiment.

5. Procedure/Algorithm:

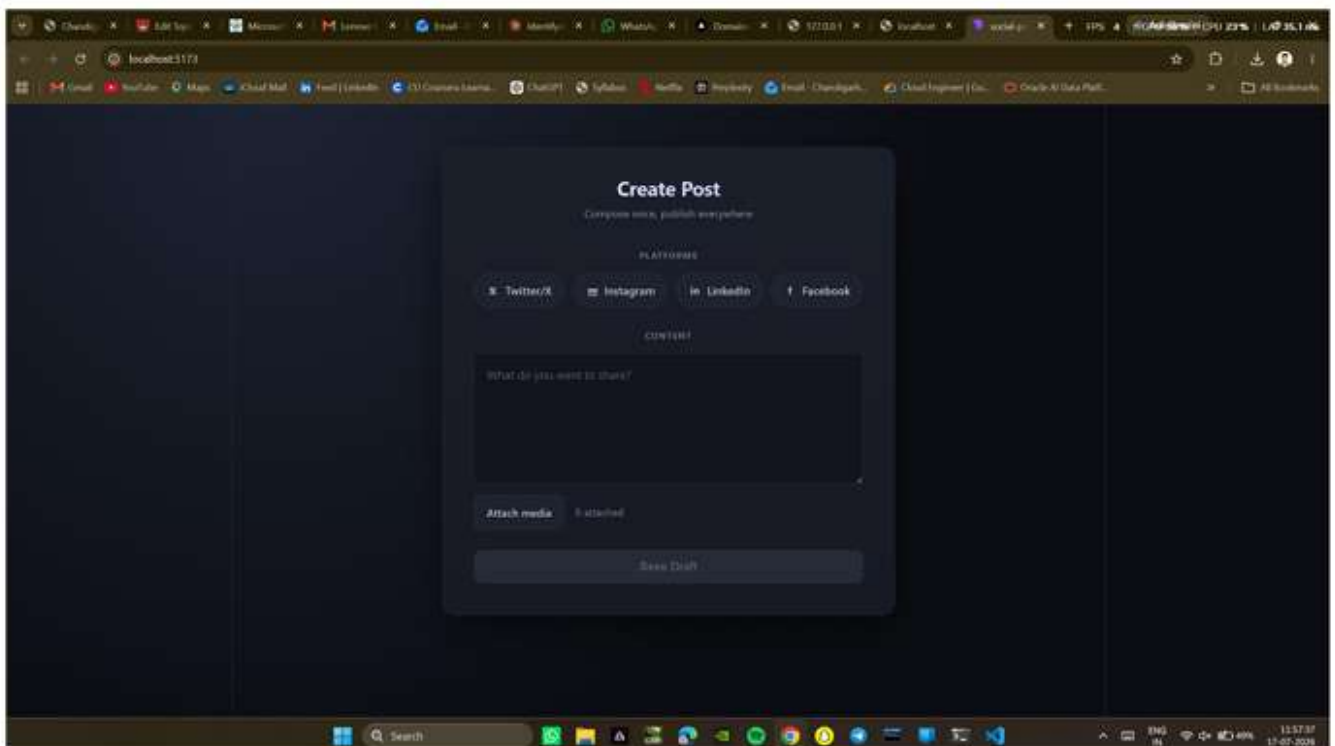
- Step 1: Define `draftCreated`, `draftUpdated`, and `draftDeleted` reducers in the Redux slice, storing drafts as `{ byId, allIds }`.
- Step 2: On Save Draft, dispatch `draftCreated` (first save) or `draftUpdated` (subsequent saves) with the current text, platforms, and media.
- Step 3: Render a Saved Drafts list from the store, showing a text preview, selected platform icons, and a relative timestamp (e.g. "5m ago") per draft.
- Step 4: On clicking a draft in the list, load its stored fields back into the composer's local state so it can continue being edited.
- Step 5: On clicking delete, dispatch `draftDeleted` and remove the draft from both `byId` and `allIds`.
- Step 6 (optional simulation): Wrap the save action in a `createAsyncThunk` with an artificial delay, so pending/fulfilled states can drive a loading indicator, mirroring how a real API call would behave.

6. Code and Output:

```
// Draft CRUD reducers
draftCreated: {
  reducer(state, action) {
    const draft = action.payload;
    state.drafts.byId[draft.id] = draft;
    state.drafts.allIds.push(draft.id);
  },
  prepare({ text, platformIds, media }) {
    return { payload: { id: nanoid(), text, platformIds, media,
      createdAt: Date.now(), updatedAt: Date.now() } };
  },
},
draftDeleted(state, action) {
  const id = action.payload;
  delete state.drafts.byId[id];
  state.drafts.allIds = state.drafts.allIds.filter((d) => d !== id);
},

// Optional simulated backend latency
```

```
export const saveDraftRemote = createAsyncThunk(
  'drafts/saveRemote',
  async (draft) => {
    await new Promise((resolve) => setTimeout(resolve, 600)); // simulated network delay
    return draft;
  }
);
```



7. Conclusion:

A complete draft lifecycle was implemented: users can save a post in progress, see it appear instantly in a Saved Drafts list, click it to resume editing, and delete it when no longer needed — all backed by normalized Redux state so each operation is $O(1)$ regardless of how many drafts exist. The optional async thunk pattern demonstrates how the same UI would behave against a real backend, without requiring one for this experiment.



8. Learning Outcomes:

- Learned to model a full CRUD lifecycle (create, update, delete) for a frontend-only entity using Redux Toolkit reducers.
- Understood how normalized state keeps draft lookup, update, and deletion at $O(1)$ instead of scanning an array.
- Learned to simulate asynchronous backend latency with `createAsyncThunk` to prototype an interaction pattern before a real API exists.